Analýza optimalizačného problému: Connect Four

[Nika Meskhi, Volodymyr Vulchyn, Volodymyr Ihnatov]

1 Analýza problému

1.1 Opis úlohy a vstupy

Connect Four je dvojhráčová strategická hra s úplnou informáciou. V štandardnej verzii hráči striedavo vkladajú žetón do jedného zo siedmich stĺpcov hracej dosky s rozmermi 7×6. Žetón sa v stĺpci umiestni na najnižšie voľné políčko. Hráč vyhráva, keď vytvorí súvislú štvoricu svojich žetónov horizontálne, vertikálne alebo diagonálne. Ak sa doska zaplní bez vytvorenia štvorice, výsledkom je remíza.

Vstupom pri jednom rozhodnutí je aktuálny stav dosky, hráč na ťahu a množina legálnych ťahov. Pri vyhodnocovaní celej stratégie sú vstupmi navyše definovaná množina súperov, pravidlá zápasu a konfigurácia experimentu. Zadanie výslovne požaduje vyhodnocovať stratégie proti viacerým súperom a sledovať výsledky pri hre za oboch začínajúcich hráčov.

V tejto práci uvažujeme štyri základné varianty rozhodovania:

1. výber ťahu pomocou parametrizovanej hodnotiacej funkcie,
2. algoritmus Minimax s alfa-beta prerezávaním,
3. výber ťahu na základe Monte Carlo simulácií,
4. Monte Carlo Tree Search (MCTS).

Jednotlivé varianty budú porovnané z hľadiska úspešnosti, výpočtovej náročnosti a schopnosti nájsť kvalitné ťahy v rôznych herných situáciách.

1.2 Typ optimalizačného problému

Za hlavnú optimalizačnú úlohu považujeme nastavenie parametrov stratégie θ tak, aby sa maximalizovala účelová funkcia $F(\theta)$ z časti 1.4. Samotné rozhodovanie stratégie v jednej partii (výber stĺpca) je vnútornou súčasťou vyhodnotenia kandidáta, a preto ho uvádzame len tam, kde mení zaradenie.

1. **Numerická alebo kombinatorická optimalizácia.**

   Zaradenie: numerická optimalizácia.

   Zdôvodnenie: Kandidát θ vzniká priradením číselných hodnôt premenným, teda váham $w_1, \ldots, w_5$, hĺbke prehľadávania d, počtu simulácií M alebo parametru prieskumu c. Nejde o výber a usporiadanie diskrétnych objektov, ktoré je typické pre kombinatorickú úlohu. Kombinatorický charakter by úloha mala len vtedy, ak by sme hľadali priamo zoznam ťahov pre každý stav hry.

2. **Rozhodovací, optimalizačný alebo kombinovaný problém.**

   Zaradenie: kombinovaný problém.

   Zdôvodnenie: Nestačí nájsť ľubovoľného kandidáta, ktorý spĺňa podmienky, ale hľadáme kandidáta s najvyššou hodnotou $F(\theta)$, teda ide o optimalizáciu. Zároveň musí byť riešenie prípustné: parametre musia patriť do zvolenej domény a stratégia musí dodržať výpočtový rozpočet a vracať len legálne ťahy. Riešením je teda kandidát s optimálnou hodnotou cieľovej funkcie, ktorý spĺňa všetky podmienky.

3. **Spojitý alebo diskrétny problém.**

   Zaradenie: spojitý problém so zmiešanými premennými.

   Zdôvodnenie: Váhy $w_i \in [-W, W]$ sú reálne čísla, a preto pri ich ladení možno hľadať smer zlepšenia v spojitom priestore. Parametre d a M sú celočíselné, takže priestor kandidátov obsahuje aj diskrétnu časť. Samotný výber ťahu v partii je diskrétny, keďže sa vyberá jeden zo siedmich stĺpcov. Ak by sme váhy vyberali z malej množiny hodnôt, napríklad $\{-2, -1, 0, 1, 2\}$, úloha by sa stala diskrétnou.

4. **Úloha s ohraničením alebo bez ohraničenia.**

   Zaradenie: úloha s ohraničením.

   Zdôvodnenie: Ohraničenia určujú, ktorý kandidát je prípustný. Ide o legálnosť ťahov, hranice parametrov, maximálnu hĺbku, maximálny počet simulácií a spoločný výpočtový rozpočet. Nelegálny výstup stratégie sa nesmie považovať za platný ťah. Ohraničenia možno riešiť kontrolou prípustnosti alebo penalizáciou v účelovej funkcii, ako je to rozpísané v časti 1.5.

5. **Jednokriteriálna alebo viackriteriálna úloha.**

   Zaradenie: jednokriteriálna úloha.

   Zdôvodnenie: Základným a jediným kritériom je priemerné skóre $F(\theta)$. Výpočtovú náročnosť v základnej formulácii nepovažujeme za samostatné kritérium, ale za ohraničenie. Aj rozšírená funkcia $F_\lambda(\theta)$ ostáva jednokriteriálna, pretože kvalitu a náklady spája do jedného čísla. Viackriteriálnou by sa úloha stala až pri hľadaní Pareto-optimálnych riešení z hľadiska úspešnosti a času rozhodovania.

6. **Deterministická alebo stochastická úloha.**

   Zaradenie: deterministická úloha pre hodnotiacu funkciu a Minimax, stochastická úloha pre Monte Carlo a MCTS.

   Zdôvodnenie: Pravidlá Connect Four sú deterministické. Ak hodnotiaca funkcia a Minimax hrajú proti deterministickým súperom a majú určené pravidlo pri rovnosti hodnôt, partie aj hodnota $F(\theta)$ sú pri rovnakých podmienkach vždy rovnaké. Pri Monte Carlo a MCTS pracuje vyhodnocovaná stratégia s náhodnými simuláciami, preto je $F(\theta)$ náhodná veličina, ktorá kolíše medzi spusteniami. Náhodnosť samotného optimalizačného algoritmu by úlohu stochastickou nerobila.

7. **Statická alebo dynamická úloha.**

   Zaradenie: statická úloha.

   Zdôvodnenie: Účelová funkcia $F(\theta)$ je definovaná na pevnej sade súperov a podmienok a v čase sa nemení, preto stačí nájsť jedno riešenie. To, že partia pozostáva zo sekvencie

ťahov a meniacich sa stavov, úlohu dynamickou nerobí, pretože dynamika úlohy nie je to isté ako dynamika hry. Dynamickou by sa stala, keby sa súperi alebo pravidlá menili a stratégiu by sme museli kontinuálne adaptovať.

8. **Biela, čierna alebo šedá skrinka.**

   Zaradenie: čierna skrinka (s prvkami šedej skrinky).

   Zdôvodnenie: Optimalizátor nepozná matematický zápis $F(\theta)$ a o kvalite kandidáta sa dozvie len po odohraní série partií. Preto ide o spätnú väzbu typickú pre čiernu skrinku. Poznáme síce pravidlá hry a štruktúru funkcie Q_w, ktorá je lineárna vo váhach, ale tie nestačia na analytické vyjadrenie úspešnosti. V súlade s prednáškou tiež rozlišujeme medzi ohodnocovacou funkciou (Q_w, $H(s)$) a cieľovou funkciou ($F(\theta)$).

1.3 Kandidátne riešenie a jeho programová reprezentácia

Kandidátom bude úplná stratégia, ktorá pre každý dosiahnuteľný stav hry vyberie legálny ťah. Uvažovať budeme tri reprezentácie: parametrizovanú hodnotiacu funkciu, Minimax a Monte Carlo Tree Search.

1.3.1 Parametrizovaná hodnotiaca funkcia

Ako základný prístup použijeme váženú hodnotiacu funkciu pre každý legálny ťah. Pre aktuálny stav s a legálny ťah a definujme

$$Q_{\boldsymbol{w}}(s,a) = w_1 f_{\text{win}}(s,a) + w_2 f_{\text{block}}(s,a) + w_3 f_{\text{center}}(s,a) + w_4 f_{\text{threat}}(s,a) + w_5 f_{\text{mobility}}(s,a).$$

Funkcie f_i merajú napríklad to, či ťah okamžite vyhrá, zablokuje bezprostrednú výhru súpera, zvýši kontrolu stredu, vytvorí potenciálne štvorice alebo zachová možnosti ďalších ťahov. Ich presná definícia musí byť pevne stanovená v implementácii.

Stratégia zvolí ťah s najvyššou hodnotou:

$$\pi_{\boldsymbol{w}}(s) = \arg\max_{a \in A(s)} Q_{\boldsymbol{w}}(s,a),$$

kde $A(s)$ je množina legálnych ťahov v stave s. Pri rovnosti skóre treba určiť reprodukovateľné pravidlo, napríklad pevné poradie stĺpcov alebo vopred nastavený generátor náhodných čísel.

1.3.2 Minimax s alfa-beta prerezávaním

Minimax je algoritmus prehľadávania herného stromu, ktorý predpokladá, že obaja hráči sa snažia dosiahnuť čo najlepší výsledok pre seba. Ťahy jedného hráča maximalizujú hodnotu stavu, zatiaľ čo ťahy súpera ju minimalizujú.

Nech $V(s,d)$ označuje hodnotu stavu s pri zostávajúcej hĺbke prehľadávania d. Potom možno rekurziu zapísať:

$$V(s,d) = \begin{cases} U(s), & \text{ak je } s \text{ terminálny,} \\ H(s), & \text{ak } d = 0, \\ \max\limits_{a \in A(s)} V(T(s,a), d-1), & \text{ak maximalizuje hráč,} \\ \min\limits_{a \in A(s)} V(T(s,a), d-1), & \text{ak minimalizuje hráč.} \end{cases}$$

Funkcia $T(s, a)$ určuje nový stav po vykonaní ťahu a. Funkcia $U(s)$ priraďuje hodnotu ukončenej partii a funkcia $H(s)$ odhaduje hodnotu neukončeného stavu. Hodnotiaca funkcia H môže využívať rovnaké pozičné črty ako funkcia $Q_{\boldsymbol{w}}$.

Terminálne stavy musia mať prednosť pred kontrolou hĺbky. Tak sa zabezpečí, že bezprostredná výhra alebo prehra nebude nahradená iba pozičným odhadom. Hodnoty výhier a prehier možno zvoliť tak, aby algoritmus uprednostnil rýchlejšiu výhru a pomalšiu prehru.

Výpočtovú náročnosť Minimaxu znižuje **alfa-beta prerezávanie**. Algoritmus udržiava hranice α a β pre najlepšie hodnoty, ktoré už boli nájdené pre maximalizujúceho a minimalizujúceho hráča. Ak sa zistí, že ďalšia vetva nemôže zmeniť výsledné rozhodnutie, jej prehľadávanie sa ukončí.

Alfa-beta prerezávanie zachováva výsledok Minimaxu pri rovnakej hĺbke prehľadávania, ale počet preskúmaných uzlov môže výrazne znížiť. Efektivita závisí aj od poradia ťahov. Preto možno najprv skúšať okamžité výhry, blokovanie súperových hrozieb a ťahy v stredových stĺpcoch.

Kandidáta pre Minimax možno reprezentovať parametrami

$$\theta_{\mathrm{MM}} = (w_1, \ldots, w_5, d),$$

kde d označuje maximálnu hĺbku prehľadávania. Pri rozšírenej formulácii možno pridať aj ďalšie parametre, napríklad spôsob zoradenia ťahov.

1.3.3 Monte Carlo simulácie

Monte Carlo prístup odhaduje hodnotu ťahu pomocou opakovaných simulácií pokračovania hry. Pre každý kandidátny ťah sa vykoná viacero simulácií od stavu, ktorý vznikne po jeho zahraní. Počas simulácie sa hra dohrá podľa vopred určenej simulačnej politiky.

Ak M_a označuje počet simulácií pre ťah a a r_j výsledok j-tej simulácie z pohľadu hodnoteného hráča, odhad hodnoty ťahu je

$$\widehat{Q}_{\mathrm{MC}}(s, a) = \frac{1}{M_a} \sum_{j=1}^{M_a} r_j,$$

kde $r_j \in \{1, 0.5, 0\}$ predstavuje výhru, remízu a prehru. Následne sa vyberie legálny ťah s najvyššou odhadnutou hodnotou.

Výsledok závisí od počtu simulácií a od spôsobu výberu ťahov počas simulácie. Jednoduchá náhodná politika je ľahko implementovateľná, ale nemusí poskytovať kvalitné odhady. Simulačnú politiku preto možno vylepšiť tak, aby uprednostňovala okamžité výhry a blokovanie bezprostredných hrozieb.

Pri rovnakom počte simulácií pre všetky legálne ťahy je porovnávanie jednoduché, ale nie vždy efektívne. Niektoré ťahy môžu byť zjavne nevýhodné, zatiaľ čo iné si zaslúžia podrobnejšie preskúmanie. Tento problém rieši selektívnejšie prehľadávanie MCTS.

1.3.4 Monte Carlo Tree Search (MCTS)

Monte Carlo Tree Search je algoritmus, ktorý postupne buduje herný strom a rozdeľuje výpočtový rozpočet medzi jeho vetvy podľa doterajších výsledkov simulácií.

Typická implementácia obsahuje štyri kroky:

1. **Výber uzla.** Zo stromu sa vyberá vetva podľa štatistík doterajších návštev a odmien.

2. **Rozšírenie stromu.** Ak to stav a pravidlá algoritmu umožňujú, pridá sa nový uzol zodpovedajúci nepreskúmanému ťahu.

3. **Simulácia.** Z vybraného uzla sa odohrá pokračovanie hry až po terminálny stav alebo po vopred definovanú hranicu simulácie.

4. **Aktualizácia.** Výsledok simulácie sa použije na aktualizáciu počtu návštev a odmien uzlov na preskúmanej ceste.

Na výber uzlov možno použiť kritérium UCB1:

$$\mathrm{UCB1}(s,a) = \overline{X}_{s,a} + c\sqrt{\frac{\ln N_s}{N_{s,a}}},$$

kde $\overline{X}_{s,a}$ je priemerná odmena po výbere ťahu a, N_s je počet návštev uzla s, $N_{s,a}$ je počet návštev príslušného ťahu a $c > 0$ určuje mieru prieskumu.

Prvý člen podporuje ťahy s dobrými doterajšími výsledkami. Druhý člen podporuje skúmanie menej navštívených možností. Pri implementácii treba správne určiť perspektívu odmien: hodnota zaznamenaná z pohľadu jedného hráča sa pri výbere ťahov súpera musí interpretovať konzistentne.

Kandidáta pre MCTS možno reprezentovať parametrami

$$\theta_{\mathrm{MCTS}} = (M, c, \rho),$$

kde M označuje počet simulácií alebo inú hranicu výpočtového rozpočtu, c je parameter prieskumu a ρ predstavuje simulačnú politiku.

V porovnaní s Minimaxom MCTS neprehľadáva všetky vetvy do rovnakej hĺbky. Výpočtový rozpočet rozdeľuje podľa doterajších výsledkov. Jeho odhady sú však závislé od náhodnosti simulácií a môžu sa meniť medzi jednotlivými spusteniami.

1.3.5 Programová reprezentácia

Parametrizovanú stratégiu možno uložiť ako vektor váh $\boldsymbol{w} = (w_1, \ldots, w_5)$. Rozšírené kandidáty môžu navyše obsahovať hĺbku Minimaxu, počet simulácií alebo parameter MCTS.

Stav dosky možno reprezentovať maticou 6×7 s hodnotami pre prázdne políčko a žetóny oboch hráčov. Alternatívne možno použiť bitboard, ktorý môže zefektívniť kontrolu výhier a vyhodnocovanie stavov.

Pre prvú verziu projektu odporúčame jednoduchú a ľahko testovateľnú reprezentáciu. Funkcie na generovanie legálnych ťahov, vykonávanie ťahov a kontrolu konca partie majú byť spoločné pre všetky porovnávané algoritmy. Tým sa zníži riziko, že rozdiely vo výsledkoch budú spôsobené odlišnou implementáciou pravidiel hry.

1.4 Účelová funkcia

Cieľom je maximalizovať úspešnosť stratégie na pevnej testovacej sade súperov a podmienok. Nech N je počet odohraných partií a výsledok partie i je $r_i \in \{1, 0.5, 0\}$ pre výhru, remízu a prehru optimalizovanej stratégie. Priemerné skóre definujeme:

$$F(\theta) = \frac{1}{N}\sum_{i=1}^{N} r_i, \qquad \max_{\theta} F(\theta).$$

Všeobecný parameter θ predstavuje kandidáta príslušnej metódy. Pre základnú hodnotiacu funkciu obsahuje váhy $\boldsymbol{w}$, pre Minimax môže obsahovať váhy a hĺbku prehľadávania a pre MCTS počet simulácií a parameter prieskumu.

Pri každej sade experimentov sa musí zaznamenať, či kandidát hral ako prvý alebo druhý hráč. Odporúča sa uviesť aj samostatnú mieru výhier, remíz a prehier, pretože rovnaké priemerné skóre môže vzniknúť z odlišných rozdelení výsledkov.

Pri Monte Carlo a MCTS treba vyhodnotenie opakovať s viacerými náhodnými semenami. Okrem priemeru je vhodné reportovať aj variabilitu výsledkov. Pri deterministickom Minimax algoritme sa zase musí zaznamenať hĺbka prehľadávania a spôsob riešenia rovnakých hodnôt ťahov.

Ak chceme zahrnúť aj výpočtovú náročnosť, možno použiť rozšírenú účelovú funkciu:

$$F_\lambda(\theta) = \frac{1}{N}\sum_{i=1}^{N} r_i - \lambda \frac{C(\theta)}{C_{\max}},$$

kde $C(\theta)$ predstavuje nameranú výpočtovú náročnosť, $C_{\max}$ je referenčná hodnota a $\lambda \geq 0$ určuje význam výpočtových nákladov. Náročnosť možno merať časom rozhodovania, počtom preskúmaných uzlov alebo počtom vykonaných simulácií.

Alternatívou je ponechať úspešnosť ako jediný cieľ a určiť spoločný maximálny čas alebo výpočtový rozpočet ako obmedzenie. Takáto formulácia uľahčuje férové porovnanie algoritmov s odlišnou vnútornou štruktúrou.

Aby sa zabránilo prispôsobeniu na testovaciu sadu, parametre sa majú optimalizovať na tréningovej sade súperov alebo partií a finálne porovnanie vykonať na oddelenej testovacej sade. Všetky metódy majú používať rovnakých súperov, rovnaké pravidlá a porovnateľný výpočtový rozpočet, v súlade s požiadavkou zadania.

1.5 Obmedzenia a kontrola prípustnosti

Základné obmedzenia sú:

1. Vybraný stĺpec musí patriť do množiny legálnych ťahov $A(s)$.
2. Do plného stĺpca nemožno vložiť ďalší žetón.
3. Hra sa ukončí po výhre alebo po zaplnení dosky.
4. Stratégia musí vždy vrátiť jeden legálny ťah, pokiaľ hra ešte neskončila.
5. Parametre kandidáta musia spĺňať zvolenú doménu, napríklad $w_i \in [-W, W]$ alebo $w_i \in \mathbb{R}$.
6. Minimax musí dodržať maximálnu hĺbku prehľadávania a stanovený výpočtový rozpočet.
7. Monte Carlo a MCTS musia dodržať maximálny počet simulácií, prípadne časový limit.

Legálnosť ťahu sa kontroluje pred jeho vykonaním. Nelegálny výstup stratégie sa nemá potichu považovať za platný ťah. Implementácia má zaznamenať chybu a buď kandidáta penalizovať, alebo použiť vopred definovaný opravný postup. Rovnaké pravidlá musia platiť pre všetky optimalizačné metódy.

Pri Minimax algoritme je dôležité správne ošetriť terminálne stavy, aby sa pokračovanie hry nevyhodnocovalo po výhre alebo po remíze. Pri MCTS treba zabezpečiť, aby

simulácie rešpektovali pravidlá hry a aby sa štatistiky návštev a odmien aktualizovali konzistentne.

Ak algoritmus prekročí povolený výpočtový rozpočet, musí byť vopred určené, ako sa takýto prípad vyhodnotí. Pri férovom experimente sa nesmie jednej metóde dodatočne poskytnúť viac času než ostatným.

1.6 Alternatívne formulácie

Možno uvažovať o viacerých alternatívach k navrhnutej základnej formulácii.

1. **Diskrétne namiesto spojitých parametrov.** Každú váhu možno vyberať z malej množiny hodnôt, napríklad $\{-2, -1, 0, 1, 2\}$. Priestor kandidátov sa zmenší a hodí sa pre kombinatorické prehľadávanie, ale jemné nastavenie stratégie môže byť obmedzené.

2. **Pravidlová stratégia namiesto váženej funkcie.** Kandidátom môže byť usporiadaný súbor pravidiel, napríklad najprv vyhraj, ak je to možné; inak zablokuj okamžitú výhru súpera; inak vyber ťah s najvyšším pozičným skóre. Takáto reprezentácia je interpretovateľná, no môže byť ťažšie optimalizovateľná.

3. **Minimax s alfa-beta prerezávaním.** Kandidátna stratégia môže používať Minimax s pevne stanovenou hĺbkou prehľadávania a parametrizovanou hodnotiacou funkciou. Alfa-beta prerezávanie znižuje počet skúmaných uzlov bez zmeny výsledku Minimaxu pri rovnakej hĺbke. Optimalizovať možno váhy hodnotiacej funkcie, hĺbku prehľadávania alebo spôsob poradia ťahov. Hlavným obmedzením je rastúci počet stavov pri zvyšovaní hĺbky.

4. **Monte Carlo simulácie.** Alternatívou je výber ťahov na základe výsledkov simulovaných pokračovaní hry. Pri každom legálnom ťahu sa vykoná určený počet simulácií a odhadne sa jeho priemerná úspešnosť. Metóda je jednoduchá na implementáciu, ale kvalita odhadu závisí od počtu simulácií a od simulačnej politiky.

5. **Monte Carlo Tree Search.** MCTS buduje herný strom selektívne a využíva štatistiky návštevnosti uzlov pri rozhodovaní, ktoré vetvy ďalej skúmať. Optimalizovať možno počet simulácií, simulačnú politiku a parameter prieskumu. Výhodou je flexibilné využitie výpočtového rozpočtu, nevýhodou variabilita odhadov a závislosť od kvality simulácií.

6. **Viacúčelová optimalizácia.** Okrem výsledného skóre možno optimalizovať aj čas rozhodovania alebo počet vyhodnotených stavov. Potom treba definovať kompromis medzi kvalitou hry a výpočtovými nákladmi, napríklad pomocou Pareto frontu alebo obmedzenia maximálneho času na ťah.

7. **Dynamická adaptácia stratégie.** Namiesto jedného pevného vektora váh by sa váhy mohli meniť podľa fázy hry, aktuálneho stavu alebo pozorovaného správania súpera. Kandidát by potom obsahoval pravidlá aktualizácie parametrov. Takáto formulácia je flexibilnejšia, ale zvyšuje dimenziu priestoru riešení a sťažuje férové vyhodnotenie.

8. **Kombinácia hodnotiacej funkcie s prehľadávaním.** Hodnotiaca funkcia môže byť použitá v listových uzloch Minimaxu alebo na usmernenie simulačnej politiky v MCTS. Takáto kombinácia môže zlepšiť kvalitu rozhodovania, ale vyžaduje dodatočné experimenty na určenie vhodných parametrov.

1.7 Plán experimentálneho porovnania

Na porovnanie algoritmov navrhujeme použiť rovnakú sadu súperov, rovnaké pravidlá hry a rovnaké podmienky vyhodnotenia. Každý kandidát bude hrať ako prvý aj ako druhý hráč. Pri metódach využívajúcich náhodnosť sa experimenty zopakujú s viacerými náhodnými semenami.

Metóda	**Optimalizované parametre**	**Merané ukazovatele**
Hodnotiaca funkcia	Váhy $w_1, \ldots, w_5$	Úspešnosť, počet výhier, remíz a prehier
Minimax	Váhy, hĺbka prehľadávania	Úspešnosť, čas rozhodovania, počet uzlov
Monte Carlo	Počet simulácií, simulačná politika	Úspešnosť, čas rozhodovania, počet simulácií
MCTS	Počet simulácií, parameter c, politika	Úspešnosť, čas rozhodovania, počet návštev uzlov

Tabuľka 1: Navrhované porovnanie rozhodovacích algoritmov.

Pri porovnaní treba rozlišovať medzi kvalitou rozhodovania a výpočtovou náročnosťou. Napríklad vyššia hĺbka Minimaxu môže zlepšiť úspešnosť, ale zároveň zvýšiť čas rozhodovania. Podobne viac simulácií pri MCTS môže zlepšiť odhad hodnôt ťahov, no predĺži rozhodovanie.

Výsledky preto odporúčame reportovať v dvoch podobách: pri rovnakom výpočtovom rozpočte a pri viacerých úrovniach rozpočtu. Tak možno zistiť nielen to, ktorý algoritmus dosahuje najvyššiu úspešnosť, ale aj ktorý využíva výpočtové zdroje najefektívnejšie.

Na zabránenie preučeniu sa parametre budú nastavovať na tréningovej sade. Finálne výsledky sa vyhodnotia na samostatnej testovacej sade, ktorá sa nepoužije pri výbere parametrov.

1.8 Predbežné zhrnutie a otvorené rozhodnutia

Ako prvú implementáciu navrhujeme vytvoriť parametrizovanú hodnotiacu funkciu a optimalizovať jej váhy. Následne túto stratégiu rozšírime o Minimax s alfa-beta prerezávaním a o Monte Carlo Tree Search. Základné Monte Carlo simulácie možno implementovať ako jednoduchší referenčný prístup na porovnanie s MCTS.

Cieľom bude zistiť, či systematické prehľadávanie herného stromu alebo simulácie pokračovania hry vedú k vyššej úspešnosti než samotná hodnotiaca funkcia.

Pri experimentoch budeme používať rovnakú testovaciu sadu súperov a vyhodnocovať všetky metódy pri hre za oboch začínajúcich hráčov. Okrem počtu výhier, remíz a prehier zaznamenáme čas rozhodovania a počet preskúmaných uzlov alebo vykonaných simulácií. Pre férové porovnanie stanovíme spoločný výpočtový rozpočet a oddelíme tréningové partie od finálneho testovania.

Pred implementáciou musí tím konkretizovať:

1. ktoré črty hodnotiacej funkcie implementuje,

2. akých súperov použije,

3. ako rozdelí tréningové a testovacie partie,
4. aký výpočtový rozpočet dostane každý kandidát,
5. akú hĺbku prehľadávania použije pre Minimax,
6. koľko simulácií a akú simulačnú politiku použije pre Monte Carlo a MCTS,
7. ktoré optimalizačné metódy porovná pri nastavovaní parametrov jednotlivých stratégií.